Pipeline Speed — Brainstorm & Evaluation

Status: Pre-decision. No implementation chosen yet. **Context:** Current pipeline is taking too long due to serial execution, full reruns, and expensive quality loops.

The Problem, Precisely

From the status blob: a "7 node" run executed 76 node steps.

- Full pipeline ran once (18 sections × content + diagram = ~36 nodes)
- QC failed → full rerun 1 (another ~36 nodes + assemble + QC)
- QC failed again → rerun 2 started (already at section_05)
- Every cycle: full assemble + full QC at the end

The compounding is: serial execution × multiple reruns × expensive QC = wall clock disaster.

Evaluation Criteria

Each idea rated 1–5 on:

- **Speed gain** — how much faster does it actually make the pipeline?
- **Quality impact** — does it risk degrading output quality?
- **Reliability** — does it introduce new failure modes?
- **Implementability** — how hard to build? (5 = trivial, 1 = major effort)
- **ROI** — speed gain per unit of implementation effort

Idea 1 — Survey Depth for First Pass (Codex)

Use the existing `survey` depth flag so the planner generates only core sections (fewer sections = fewer nodes). Already supported in UI and planner prompt.

What it does: Reduces the number of sections generated from ~18 to maybe 8–10. Since nodes scale with sections, this linearly reduces all downstream work.

What it doesn't do: It doesn't fix the serial execution or the rerun loop — it just makes each cycle smaller.

Criterion	Score	Notes
Speed gain	3/5	40–50% faster, scales with section count
Quality impact	2/5	Shorter material. Less coverage per session. Trade-off.
Reliability	5/5	Already works, zero new risk
Implementability	5/5	Config/UI toggle, already exists
ROI	4/5	High return for zero effort

Verdict: Good quick win for demo/testing. Not the right answer for production output because it degrades coverage. Should be the default for `draft` mode only.

Idea 2 — Kill the Rerun Loop (MAX_QUALITY_RERUNS=0) (Codex)

Set the quality checker to run once but never trigger a rerun cycle.
You get a quality report but the pipeline delivers the first-pass output regardless.

What it does: Eliminates the rerun spiral entirely. One pass, one QC, done.

What it doesn't do: Doesn't fix quality issues — it just stops trying to fix them. A section that failed QC ships as-is.

Criterion	Score	Notes
Speed gain	5/5	Eliminates the entire rerun loop. Biggest single win.
Quality impact	1/5	Failed sections ship unrepaired. Unacceptable for production.
Reliability	4/5	Predictable — always terminates.
Implementability	5/5	One env var change
ROI	2/5	High speed, unacceptable quality cost

Verdict: Good for internal testing and development. Never for production output. Could be the backend of `draft` mode — fast, rough, ship it.

Idea 3 — Reduce Node-Level Retries (Codex)

Currently each node retries 3× with exponential backoff on transient failure.
Reduce MAX_RETRIES to 1 or 2 and shrink RETRY_BASE_DELAY.

What it does: Eliminates seconds of unnecessary waiting on nodes that would have succeeded on first retry.

What it doesn't do: Does not fix the rerun loop or serial execution.

Criterion	Score	Notes
Speed gain	2/5	Marginal — only helps on transient failures
Quality impact	3/5	Small risk: fewer retries means slightly more QC failures
Reliability	3/5	More first-pass failures bubble up to QC rerun
Implementability	5/5	Two config values
ROI	3/5	Tiny effort, modest gain

Verdict: Do it regardless. Two-line change with no downside at reasonable settings (MAX_RETRIES=1, short delay). Should not be relied on as a speed strategy.

Idea 4 — Per-Request Mode: draft | balanced | strict (Codex)

A mode field on the generation request that controls the quality/speed trade-off:

- `draft` → survey depth + no reruns + skip diagram LLM
- `balanced` → standard depth + max 1 rerun + targeted node reruns
- `strict` → current behaviour, unlimited reruns

What it does: Gives the product a clean shape for different use cases. Development/preview uses `draft`. Production output uses `balanced` or `strict`. Also serves the "two-stage UX" idea — generate draft first, then upgrade.

What it doesn't do: Does not fix the underlying serial execution — it just controls how many times you do it.

Criterion	Score	Notes
Speed gain	4/5	draft mode is very fast. balanced is 2–3× faster than strict.
Quality impact	4/5	Quality is controlled by mode choice, not compromised globally
Reliability	4/5	Clear contract per mode
Implementability	3/5	Needs request DTO change, mode-aware orchestrator logic
ROI	5/5	Clean product shape, moderate effort, major UX improvement

Verdict: The best structural idea on the list. Should be built. This is the frame that most other ideas slot into (parallel = balanced/strict, skip diagrams = draft, etc.).

Idea 5 — Targeted Node Reruns (Codex + Praxis)

Instead of rerunning content + diagram + code for every failed section,
identify what specifically failed and rerun only that node.

SVG failed only → rerun DiagramGenerator for that section
Code failed only → rerun CodeGenerator for that section
Prose coherence failed → rerun ContentGenerator only

What it does: Cuts the rerun workload to roughly 1/3 of current on typical QC failure patterns (most failures are isolated to one node type).

What it doesn't do: Does not fix serial execution within a rerun.

Criterion	Score	Notes
Speed gain	4/5	Reruns shrink dramatically in typical failure cases
Quality impact	5/5	Fixes only what's broken — other nodes unaffected
Reliability	4/5	Requires accurate failure attribution from QC
Implementability	3/5	QC must emit node-specific failure tags, not just section IDs
ROI	5/5	High gain, reasonable effort

Verdict: Should be built. The key requirement is that QualityChecker's output schema includes a `failed_node_type` field per failure, not just `failed_section_id`. That is a schema change in quality_checker.py,

then orchestrator reads it.

Idea 6 — Inline QC During Generation (Praxis)

Instead of assembling everything first and then running a full QC at the end,
run a lightweight per-section check immediately after each section is generated.
Only run the full QC once at the very end for document-level checks.

What it does: Catches bad sections early — before the expensive assemble step. A section that fails inline QC is rerun immediately, before the pipeline has moved on. This prevents the current pattern of: generate all 18 sections, assemble, QC fails on 6, rerun all 6, reassemble, QC again.

What it doesn't do: Inline QC cannot catch cross-section consistency issues (e.g. concept introduced in section 12 but referenced in section 7). Those still need the full final QC.

Two-tier QC model:

```
Section QC (inline, cheap):
  - Structural checks: has heading, has hook, no empty blocks, SVG valid
  - Content checks: no raw LaTeX, no undefined symbols, min length
  - Triggered: immediately after each ContentGenerator + DiagramGenerator call

Document QC (full, expensive, once):
  - Cross-section consistency
  - LLM coherence check
  - Difficulty progression
  - Triggered: once after all sections pass inline QC
```

Criterion	Score	Notes
Speed gain	5/5	Eliminates late-stage rerun spirals almost entirely
Quality impact	5/5	Catches more issues earlier, not fewer
Reliability	4/5	Inline QC must be fast and not have false positives
Implementability	3/5	Requires splitting QC into two tiers, plumbing inline QC into section generation
ROI	5/5	Highest reliability gain per unit effort on this list

Verdict: The best quality-preserving speed improvement available. Should be the primary architectural change. Works especially well combined with parallelization.

Idea 7 — Parallelise Section Generation (Codex + Praxis)

Currently sections are generated serially: section_01 finishes, then section_02 starts. But each ContentGenerator call only needs the section spec and the learner profile — it has no dependency on other sections.

Run all ContentGenerator calls concurrently with bounded concurrency (e.g. max 5 parallel), then run DiagramGenerator calls concurrently for completed sections as they finish.

What it does: Reduces wall clock time from O(n_sections) to roughly O(1) × average_section_time with bounded parallelism. 18 serial sections at 8s each = 144s. 5 parallel = ~30s.

What it doesn't do: Does not fix QC reruns. Does not fix quality issues. Parallelism makes a fast pipeline faster — it does not fix a broken pipeline.

Criterion	Score	Notes
Speed gain	5/5	3–5× wall clock reduction, biggest raw speed gain available
Quality impact	5/5	Output is identical — just generated concurrently
Reliability	3/5	Adds concurrency complexity. Rate limits become a real concern.
Implementability	2/5	asyncio.gather + semaphore + error handling + LLM rate limit management
ROI	3/5	Big gain but meaningful implementation cost and new failure modes

Verdict: The largest single speed gain available but the most complex to implement correctly. Rate limiting must be handled carefully — firing 18 LLM calls simultaneously will hit API rate limits and degrade to serial anyway, plus errors. Implement after targeted reruns and inline QC are stable. Use bounded concurrency (asyncio.Semaphore with limit of 3–5), not unbounded.

Idea 8 — Deferred Diagram Tags (Praxis)

Rather than blocking section assembly on DiagramGenerator completion, insert placeholder tags during first pass and fill them in a separate post-processing step — potentially with a different, faster model.

What it does: Unblocks the main content pipeline from waiting for SVG generation. Content for all sections can be generated, assembled, and delivered to the learner while diagrams are filled in asynchronously or on demand.

Two-phase delivery:

```
Phase 1 (fast):
  Content only — text, definitions, worked examples, code.
  Diagrams replaced with <diagram-pending id="d01" desc="..."/> tags.
  Delivered to learner in ~40% of current time.

Phase 2 (async):
  DiagramFiller model processes pending tags.
  Fills them with actual SVGs.
  Panel updates as diagrams complete (progressive enhancement).
```

The "filler model" concept (Praxis): The diagram filler does not need to be the same model as content generation. A faster/cheaper model given only the diagram description and the design system CSS can produce the SVG. The content model never has to wait for it.

Criterion	Score	Notes
Speed gain	4/5	Time-to-first-content drops dramatically
Quality impact	4/5	Diagrams still generated, just asynchronously
Reliability	3/5	Progressive UI updates add frontend complexity
Implementability	3/5	Tag system + async filler + frontend polling/websockets
ROI	4/5	Good gain, manageable complexity if websockets already exist

Verdict: Excellent UX improvement — learner sees content fast and diagrams load in progressively. Requires frontend support for streaming updates. Works well as a feature on top of parallelization.

Idea 9 — Model Routing by Task Type (Praxis)

Different pipeline nodes have different requirements:

- CurriculumPlanner: needs reasoning, low latency tolerance
- ContentGenerator (prose): needs quality, medium latency
- ContentGenerator (definitions, hooks): short, fast, cheap model fine
- DiagramGenerator: structured output, fast model sufficient
- QualityChecker (mechanical checks): no LLM needed at all

- QualityChecker (coherence): needs reasoning, expensive

Route each node type to the appropriate model tier:

Node	Current	Proposed
CurriculumPlanner	claude-sonnet	claude-sonnet (keep, high stakes)
ContentGenerator/prose	claude-sonnet	claude-sonnet (keep, quality matters)
ContentGenerator/short	claude-sonnet	claude-haiku (switch, short blocks)
DiagramGenerator	claude-sonnet	claude-haiku (switch, structured SVG)
QC mechanical	claude-sonnet	none (rules) (eliminate LLM entirely)
QC coherence	claude-sonnet	claude-sonnet (keep, needs reasoning)
DiagramFiller	—	claude-haiku (new, fills tags)

What it does: Cuts LLM cost by ~40–60% and reduces latency on fast-model nodes (haiku is 3–5× faster than sonnet at most tasks).

What it doesn't do: Does not fix serial execution — but makes each serial step faster.

Criterion	Score	Notes
Speed gain	3/5	Haiku nodes are faster; not as dramatic as parallelism
Quality impact	3/5	Risk: haiku may produce lower-quality diagrams and short blocks
Reliability	4/5	Model routing is clean to implement
Implementability	4/5	Provider factory already supports this; add model config per node
ROI	4/5	Cost reduction + moderate speed gain + already-designed provider abstraction

Verdict: Should be built. Low risk because you can A/B test haiku vs sonnet per node type before committing. QC mechanical checks eliminating the LLM call entirely is the cleanest win here — pure rules run in milliseconds.

Idea 10 — Two-Stage UX: Draft First, Polish on Demand (Codex + Praxis)

Deliver a "good first presentation" to the learner quickly. Then offer a "polish this" or "expand with diagrams" action that upgrades the draft in the background while the learner is already reading.

What it does: Decouples time-to-first-content from time-to-final-quality. The learner gets something useful in 30–60 seconds. The polished version arrives while they are reading.

UX flow:

```
[0s]   Diagnostic complete. "Building your reading..."
[30s]  Draft delivered. Learner starts reading.
       "Diagrams loading..." badges on pending sections.
[90s]  Diagrams fill in progressively.
[120s] "Your reading is fully ready" notification.
```

This is not a quality compromise — the content is the same. Only the delivery sequence changes.

Criterion	Score	Notes
Speed gain	5/5	Time-to-first-content is the metric that matters for UX
Quality impact	5/5	Final output identical, just delivered progressively
Reliability	3/5	Progressive delivery requires frontend streaming support
Implementability	2/5	Needs websockets/SSE, frontend progressive rendering, draft schema
ROI	4/5	Best UX improvement but highest frontend complexity

Verdict: The right long-term UX shape. Build it after the backend speed improvements are stable. Requires deferred diagram tags (Idea 8) as a prerequisite.

Idea 11 — Fix the Progress UI Accuracy (Codex)

Currently the progress bar shows "X of 7 nodes" while the pipeline is actually executing 70+ node steps. This makes the pipeline look slower than it is and is misleading about what is happening.

What it does: Makes the UX honest. Learner sees "Generating section 5 of 18" not "Step 3 of 7." Faster pipelines feel faster when progress is accurately reported.

Criterion	Score	Notes
Speed gain	0/5	Does not make anything faster
Quality impact	0/5	Not relevant
Reliability	5/5	No risk
Implementability	4/5	UI change only
ROI	3/5	Low effort, meaningful UX improvement

Verdict: Do it regardless, in parallel with any backend work. It is a one-hour UI fix that makes every other improvement feel more impactful.

Master Comparison Table

Idea	Speed	Quality	Reliability	Implementability	ROI	Priority
1. Survey depth	3	2	5	5	4	Now (draft mode only)
2. Kill rerun loop	5	1	4	5	2	Dev/test only
3. Reduce node retries	2	3	3	5	3	Now (trivial)
4. draft|balanced|strict mode	4	4	4	3	5	**Phase 1**
5. Targeted node reruns	4	5	4	3	5	**Phase 1**
6. Inline QC + deferred doc QC	5	5	4	3	5	**Phase 1**
7. Parallelise sections	5	5	3	2	3	Phase 2
8. Deferred diagram tags	4	4	3	3	4	Phase 2
9. Model routing by task	3	3	4	4	4	**Phase 1**
10. Two-stage UX (draft→polish)	5	5	3	2	4	Phase 3
11. Fix progress UI	0	0	5	4	3	Now (trivial)

Recommended Staged Approach

Right Now (Zero Code — Config + Trivial UI)

- Idea 3: Drop MAX_RETRIES to 1
- Idea 11: Fix progress bar to show per-section progress
- Idea 1: Set `survey` as default for first-time generation

These three changes together reduce visible slowness and perceived waiting without touching the architecture.

Phase 1 (Architectural — High ROI, Moderate Effort)

These four ideas compose naturally:

1. **Idea 4** — Add draft|balanced|strict mode to request DTO
2. **Idea 5** — QC emits node-specific failure types; orchestrator reruns only the right node
3. **Idea 6** — Split QC into inline (per section) + document (once at end)
4. **Idea 9** — Route DiagramGenerator and short-block ContentGenerator to haiku

Expected combined result: pipeline in balanced mode runs in roughly 30–40% of current time. Reruns target only the broken node, not the whole section.

Phase 2 (Performance — High Speed, Higher Complexity)

1. **Idea 7** — Parallelise ContentGenerator calls with bounded concurrency
2. **Idea 8** — Deferred diagram tags with async filler model

Expected result: wall clock time drops to 20–25% of baseline for a full-depth run.

Phase 3 (UX — Progressive Delivery)

1. **Idea 10** — Two-stage UX: draft delivered first, diagrams fill in progressively

This requires Phase 2 (deferred tags) as a prerequisite and frontend streaming support.

The Single Most Impactful Change

If forced to pick one: **Idea 6 (inline QC + deferred document QC)**.

The reason: the current failure pattern is "generate everything, discover failures late, redo everything." Inline QC catches failures at generation time, repairs them immediately, and the document-level QC runs once on a clean set of sections. This eliminates the rerun spiral almost entirely, with no quality loss.

It also composes with everything else — parallelism + inline QC is better than either alone. Model routing + inline QC means the cheap-model nodes fail fast and are caught immediately.

End of Brainstorm. No implementation chosen. Decide on phase and mode, then implement.